Aula 6: A Árvore Genealógica - Herança (Generalização)



Projeto: FiapRide (Sistema de Mobilidade Urbana) + Seu Projeto Pessoal

Foco: Superclasses, Subclasses, Reaproveitamento de Código e Clean Code

🎯 Objetivo da Aula

No nosso sistema FiapRide, nós já temos a classe `Veiculo` que possui `placa` e `modelo`. Mas, pense no mundo real: a Uber, a 99 e outros apps não oferecem apenas carros comuns. Eles oferecem Carros, Motos, e até Motos elétricas!

Se fôssemos criar uma classe para cada um do zero, repetiríamos os mesmos atributos (`placa` , `modelo`) várias vezes. Hoje, você vai aprender a usar a **Herança**. Vamos criar uma "Classe Mãe" genérica e "Classes Filhas" específicas que herdam tudo o que a mãe tem, mas adicionam suas próprias características.

Conceitos Chave:

- **Superclasse (Classe Mãe):** A classe mais genérica (Ex: `Veiculo`).
- **Subclasse (Classe Filha):** A classe mais específica (Ex: `Carro` , `Moto`).
- **A Regra do "É UM":** A prova de fogo para saber se você pode usar herança.

📖 A Teoria: O Teste do "É UM"

Na programação Orientada a Objetos, a Herança serve para expressar uma relação de **Generalização/Especialização**.

- **Teste Lógico:**
- Um Carro **É UM** Veículo? Sim! Então `Carro` pode herdar de `Veiculo`.
- Uma Moto **É UM** Veículo? Sim! Então `Moto` pode herdar de `Veiculo`.
- Um Carro **É UM** Motor? Não! O Carro **TEM UM** Motor (Isso é Associação, Aula 05). NUNCA use herança se a resposta for não!

Quando uma classe filha herda de uma mãe, no Java usamos a palavra `extends` (estende). A filha ganha acesso a todos os atributos e métodos (que não sejam puramente privados sem getters) da mãe!

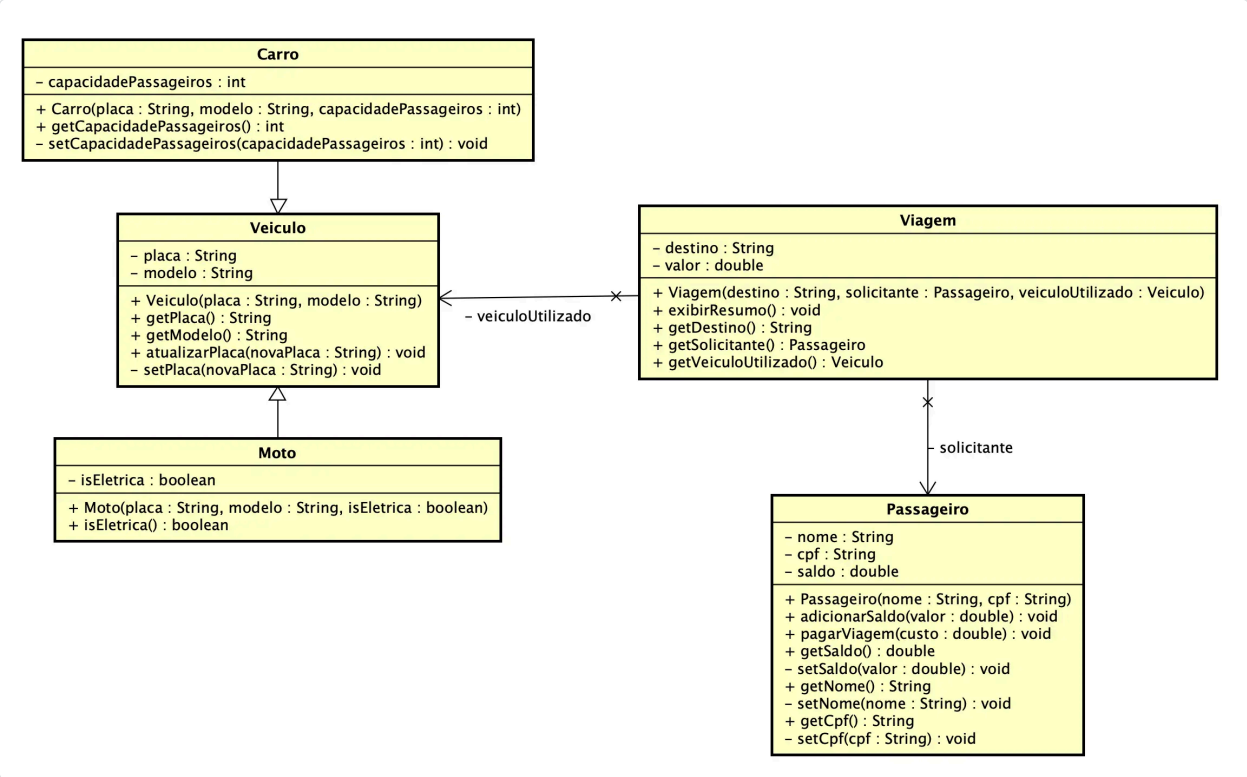
🛠️ Parte 1: Modelagem Visual (Astah)

Vamos desenhar a nossa árvore genealógica.

1. Abra o **Astah** no seu projeto FiapRide.
2. Você já tem a classe `Veiculo` pronta.
3. Crie duas **novas classes** abaixo dela: `Carro` e `Moto`.
4. Adicione características únicas nas filhas:

- Em `Carro`: adicione `- capacidadePassageiros: int`
- Em `Moto`: adicione `- isEletrica: boolean`

1. **A Mágica do UML:** Selecione a ferramenta **Generalization** (é uma linha contínua com uma seta **branca e vazada** na ponta).
2. Clique na filha (`Carro`) e arraste até a mãe (`Veiculo`). A seta deve apontar PARA A MÃE. Faça o mesmo com a `Moto`.



Nota: No diagrama, a filha NÃO precisa repetir os atributos da mãe. A seta já diz que ela tem tudo.

💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Vamos codificar nossas subclasses. Mas preste muita atenção: como a mãe (`Veiculo`) tem um Construtor obrigatório (exige placa e modelo), **as filhas são obrigadas a respeitar essa regra**!

Passo 1: A Classe Filha `Carro`

Crie a classe `Carro` no pacote `br.com.fiapride.model` .

```
package br.com.fiapride.model;

// A palavra 'extends' faz a mágica da herança acontecer!
public class Carro extends Veiculo {

    // Atributo exclusivo do Carro (a mãe Veiculo não tem isso)
    private int capacidadePassageiros;

    // CONSTRUTOR DA FILHA
    public Carro(String placa, String modelo, int capacidadePassageiros) {
        // 'super' significa SUPERCLASSE (Mãe).
        // Estamos repassando a placa e o modelo para o construtor da mãe resolver!
        super(placa, modelo);

        // E aqui o Carro resolve o que é dele:
```

```
        this.setCapacidadePassageiros(capacidadePassageiros);
    }

    // Getters e Setters específicos do Carro
    public int getCapacidadePassageiros() {
        return this.capacidadePassageiros;
    }

    private void setCapacidadePassageiros(int capacidadePassageiros) {
        this.capacidadePassageiros = capacidadePassageiros;
    }
}
```

Passo 2: A Classe Filha `Moto`

Crie a classe `Moto` no mesmo pacote.

```
package br.com.fiapride.model;

public class Moto extends Veiculo {

    private boolean isEletrica;

    public Moto(String placa, String modelo, boolean isEletrica) {
        super(placa, modelo); // Repassa para a mãe
        this.isEletrica = isEletrica;
    }

    public boolean isEletrica() {
        return this.isEletrica;
    }
}
```

Passo 3: O Teste Genético (`SistemaPrincipal`)

Vá para o seu `main` e vamos provar que a herança funciona.

```
package br.com.fiapride.main;

import br.com.fiapride.model.Carro;
import br.com.fiapride.model.Moto;

public class SistemaPrincipal {
    public static void main(String[] args) {

        System.out.println("--- FIAPRIDE: Teste de Frota ---");

        // 1. Instanciando os objetos específicos
        int quantidadePassageiro = 4;
        Carro uberX = new Carro("ABC-1234", "Chevrolet Onix", quantidadePassageiro);
        Moto mottu = new Moto("ABC-9999", "Caloi City", true);
```

```
        // 2. O TESTE: Um Carro consegue acessar getPlaca() da mãe?
        // Sim! Note que nós não escrevemos getPlaca() dentro da classe Carro. Ele herdou!
        System.out.println("Carro modelo: " + uberX.getModelo() + " | Placa: " + uberX.getPlaca());
        System.out.println("Vagas para passageiros: " + uberX.getCapacidadePassageiros());

        System.out.println("\nMoto modelo: " + mottu.getModelo() + " | Placa: " + mottu.getPlaca());

        // Se a moto é elétrica, ela avisa:
        if (mottu.isEletrica()) {
            System.out.println("Atenção: Esta moto é elétrica e atinge maiores velocidades!");
        }
    }
}
```

🧠 Checklist: Clean Code (Composição vs. Herança)

- [ ] **Cuidado com a Herança Gula:** Iniciantes tendem a usar herança apenas para não ter que digitar código de novo. Isso é um erro!
- [ ] **A Regra de Ouro:** Se você se perguntar "A classe A é um tipo de B?" e a resposta for "Não, ela apenas usa uma parte de B", **NÃO USE HERANÇA**. Use a Associação (Aula 05).
- [ ] Herança cria **um acoplamento fortíssimo** (a filha fica eternamente dependente da mãe). Só use se a relação "É UM" for absoluta.

⚡ Microdesafio Pessoal: Crie seus Herdeiros

Volte para a classe do **seu** projeto pessoal.

Sua Missão:

1. Olhe para a sua classe principal (Ex: `Smartphone`, `Pet`, `Garrafa`). Ela será a sua **Superclasse**.
2. Pense em 2 subtipos ou especializações dessa classe.
   - *Ex: Se você tem `Pet`, crie as filhas `Cachorro` e `Gato`.*
   - *Ex: Se você tem `Smartphone`, crie as filhas `Android` e `Iphone`.*
3. Crie essas duas classes no Eclipse usando a palavra `extends`.
4. Dê a cada classe filha pelo menos UM atributo exclusivo (algo que um `Cachorro` tem que um `Gato` não tem).
5. Ajuste o construtor usando o `super()`.
6. Teste no seu `main` pessoal!

🧠 Pergunta de Reflexão

"No nosso código, a mãe `Veiculo` possui os atributos `placa` e `modelo` como `private`. Quando o `Carro` herda de `Veiculo`, ele recebe esses atributos, mas o código dentro de `Carro` NÃO consegue fazer `this.placa = "ABC"`. Ele é obrigado a usar o `super()` ou o `setPlaca()`.

Por que o Java não deixa a filha alterar as variáveis privadas da mãe diretamente? Qual o princípio das aulas passadas que isso está protegendo?"

📤 Finalizando (GitHub)

```
git add .
git commit -m "feat: implementa heranca com superclasses e subclasses"
git push origin main
```

Bons estudos!

